step4_all_model_evaluation

February 6, 2026

[75]:
```
import shap
import pandas as pd
import numpy as np
import os
import joblib
from sklearn.metrics import accuracy_score, classification_report,␣
 ↪confusion_matrix, roc_auc_score, roc_curve
import matplotlib.pyplot as plt
import xgboost as xgb
import seaborn as sns
```

[76]:
```
data_dir = '../../data'
input_dir = f'{data_dir}/step3_model_outputs'
output_dir = f'{data_dir}/step4_evaluation_outputs'

file_paths = {
    'X_test_scaled_df': f'{input_dir}/step3_all_x_test_scaled',
    'y_test_df': f'{input_dir}/step3_all_y_test',
}
```

[77]:
```
#    DataFrame
dataframes = {}

#         pd.read_paqcet
for key, file_path in file_paths.items():
    if not os.path.exists(file_path):
        print(f" :    '{file_path}'          ")
        continue

    try:
        # lineterminator='\n'
        df = pd.read_parquet(file_path)
        dataframes[key] = df
        print(f"'{key}'      DataFrame  : {df.shape}")
    except Exception as e:
        print(f"  : '{file_path}'              : {e}")
```

```
'X_test_scaled_df'      DataFrame  : (12980, 289)
```

```
'y_test_df'       DataFrame  : (12980, 1)
```

[78]:
```
#
rf_model = joblib.load( f'{input_dir}/all_rf_model.joblib')
xgb_model = joblib.load( f'{input_dir}/all_xgb_model.joblib')
lgb_model = joblib.load( f'{input_dir}/all_lgb_model.joblib')
```

[79]:
```
#data
X_test_scaled_df = dataframes['X_test_scaled_df']
y_test = dataframes['y_test_df']
```

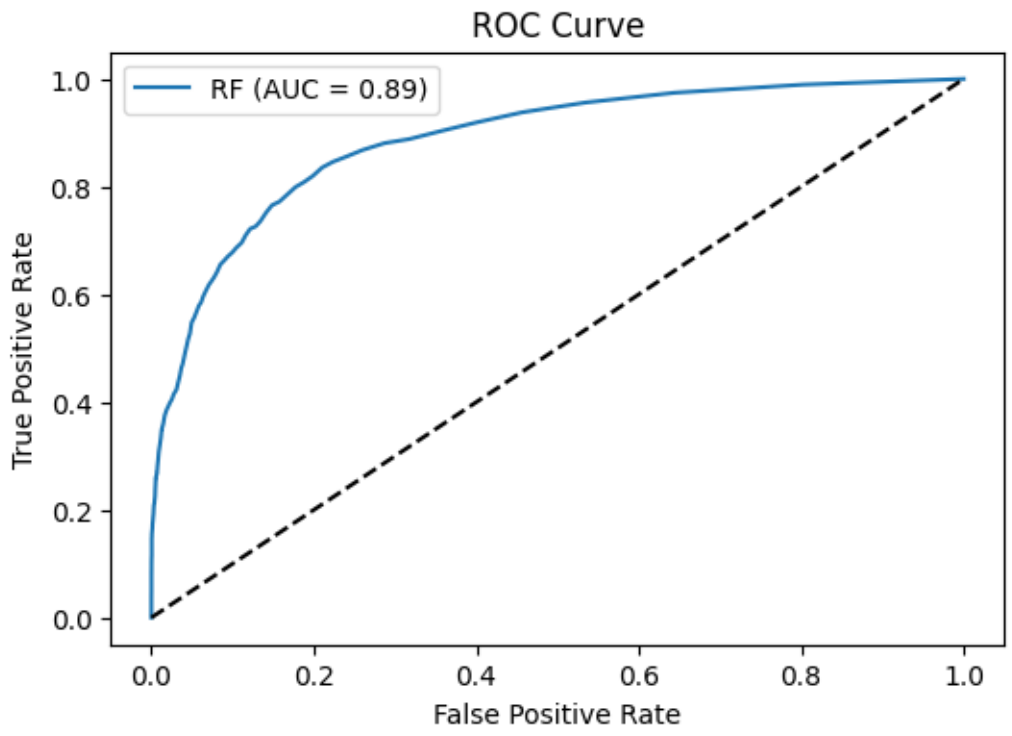
[80]:
```
#
rf_y_pred = rf_model.predict(X_test_scaled_df)
xgb_y_pred = xgb_model.predict(X_test_scaled_df)
lgb_y_pred = lgb_model.predict(X_test_scaled_df)

print("---     RF ---")
#   (Accuracy)
print(f"Accuracy: {accuracy_score(y_test, rf_y_pred):.4f}")

print("---     XGB ---")
#   (Accuracy)
print(f"Accuracy: {accuracy_score(y_test, xgb_y_pred):.4f}")

print("---     LightGBM ---")
#   (Accuracy)
print(f"Accuracy: {accuracy_score(y_test, lgb_y_pred):.4f}")

#       (    F1   )
print("\nClassification Report:")
print(classification_report(y_test, rf_y_pred))
print(classification_report(y_test, xgb_y_pred))
print(classification_report(y_test, lgb_y_pred))
```

```
---     RF ---
Accuracy: 0.9155
---     XGB ---
Accuracy: 0.9061
---     LightGBM ---
Accuracy: 0.8985

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.97      0.95     11654
           1       0.63      0.41      0.50      1326

    accuracy                           0.92     12980
```

```
   macro avg       0.78      0.69      0.73     12980
weighted avg       0.90      0.92      0.91     12980

              precision    recall  f1-score   support

           0       0.95      0.94      0.95     11654
           1       0.54      0.58      0.56      1326

    accuracy                           0.91     12980
   macro avg       0.74      0.76      0.75     12980
weighted avg       0.91      0.91      0.91     12980

              precision    recall  f1-score   support

           0       0.95      0.93      0.94     11654
           1       0.50      0.59      0.54      1326

    accuracy                           0.90     12980
   macro avg       0.73      0.76      0.74     12980
weighted avg       0.91      0.90      0.90     12980
```

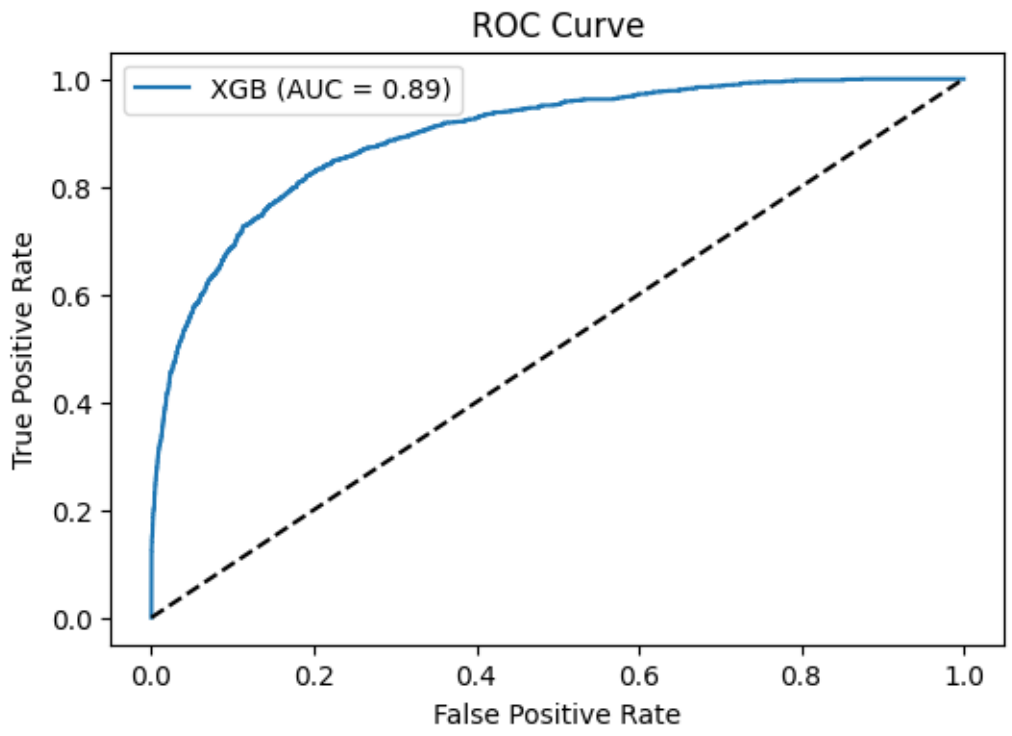
[81]:
```python
#AUC
rf_y_probs = rf_model.predict_proba(X_test_scaled_df)[:, 1]

auc_score = roc_auc_score(y_test, rf_y_probs)
print(f"ROC-AUC Score: {auc_score:.4f}")

fpr, tpr, thresholds = roc_curve(y_test, rf_y_probs)

plt.figure(figsize=(6, 4))
plt.plot(fpr, tpr, label=f'RF (AUC = {auc_score:.2f})')
plt.plot([0, 1], [0, 1], 'k--') #
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```

```
ROC-AUC Score: 0.8869
```



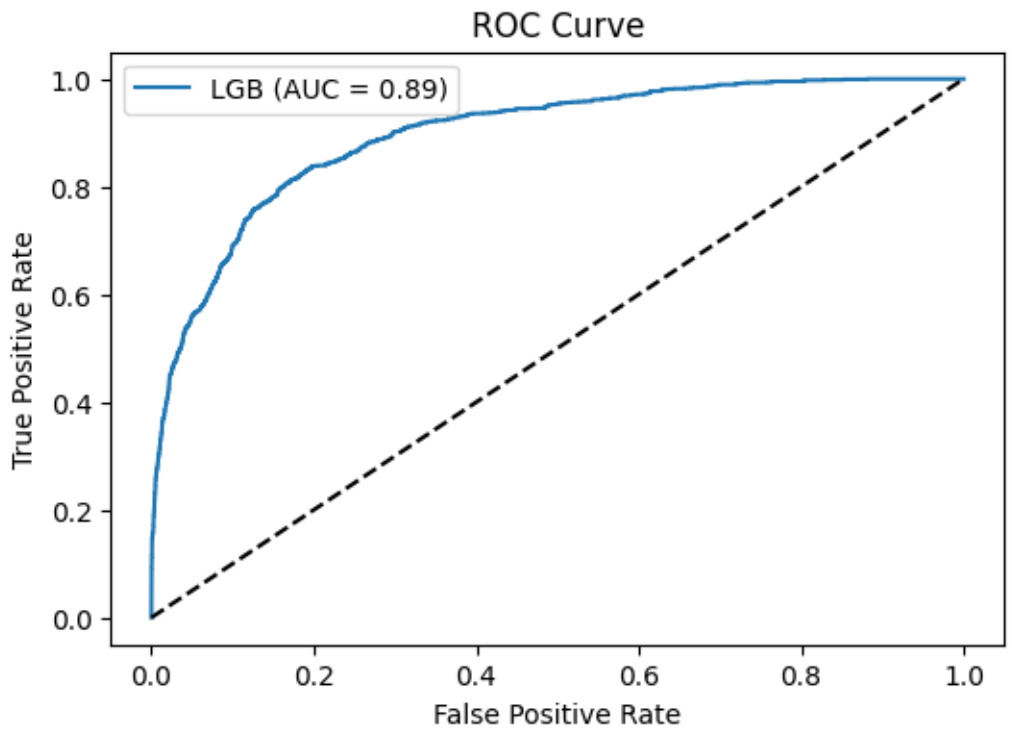
[82]:
```
#AUC
xgb_y_probs = xgb_model.predict_proba(X_test_scaled_df)[:, 1]

auc_score = roc_auc_score(y_test, xgb_y_probs)
print(f"ROC-AUC Score: {auc_score:.4f}")

fpr, tpr, thresholds = roc_curve(y_test, xgb_y_probs)

plt.figure(figsize=(6, 4))
plt.plot(fpr, tpr, label=f'XGB (AUC = {auc_score:.2f})')
plt.plot([0, 1], [0, 1], 'k--') #
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```

```
ROC-AUC Score: 0.8940
```

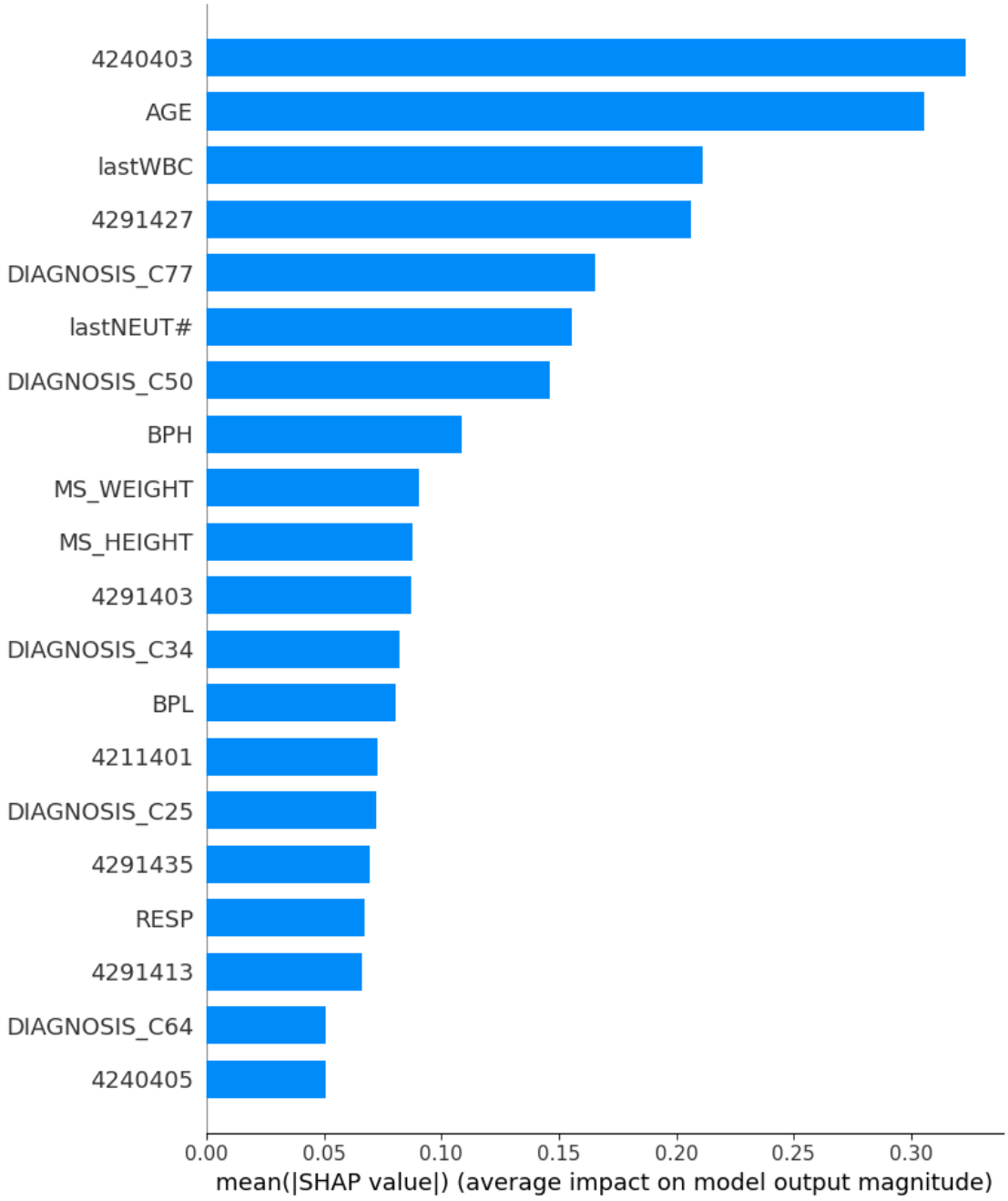


[83]:
```
lgb_y_probs = lgb_model.predict_proba(X_test_scaled_df)[:, 1]
print(f"ROC-AUC Score: {roc_auc_score(y_test, lgb_y_probs):.4f}")

fpr, tpr, thresholds = roc_curve(y_test, lgb_y_probs)

plt.figure(figsize=(6, 4))
plt.plot(fpr, tpr, label=f'LGB (AUC = {auc_score:.2f})')
plt.plot([0, 1], [0, 1], 'k--') #
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```

```
ROC-AUC Score: 0.8961
```



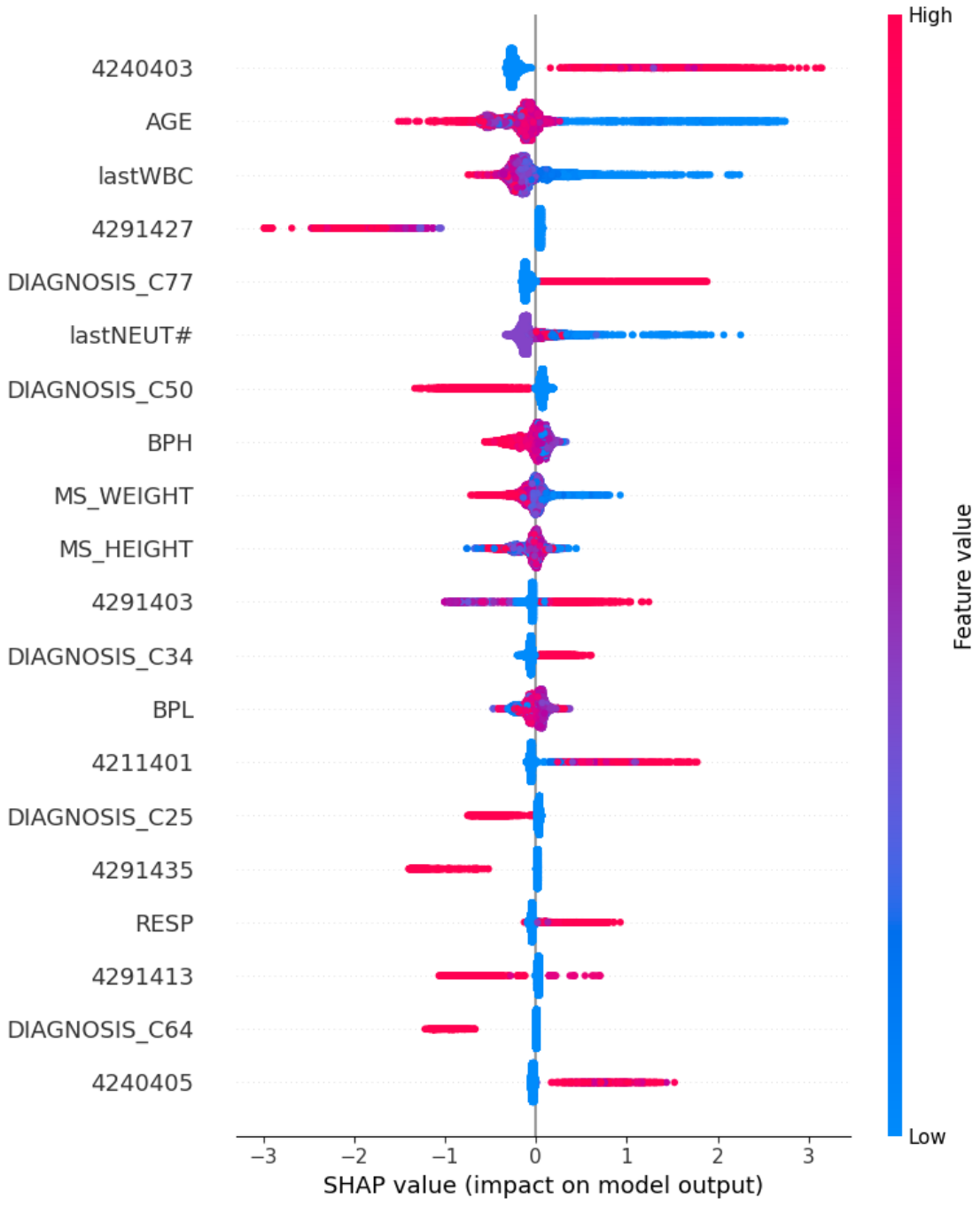
[86]:
```
shap.summary_plot(shap_values, X_test_scaled_df, plot_type="bar")
```



[85]:
```
#       Explainer
explainer = shap.TreeExplainer(xgb_model)

#         SHAP
shap_values = explainer.shap_values(X_test_scaled_df)

#SHAP
```

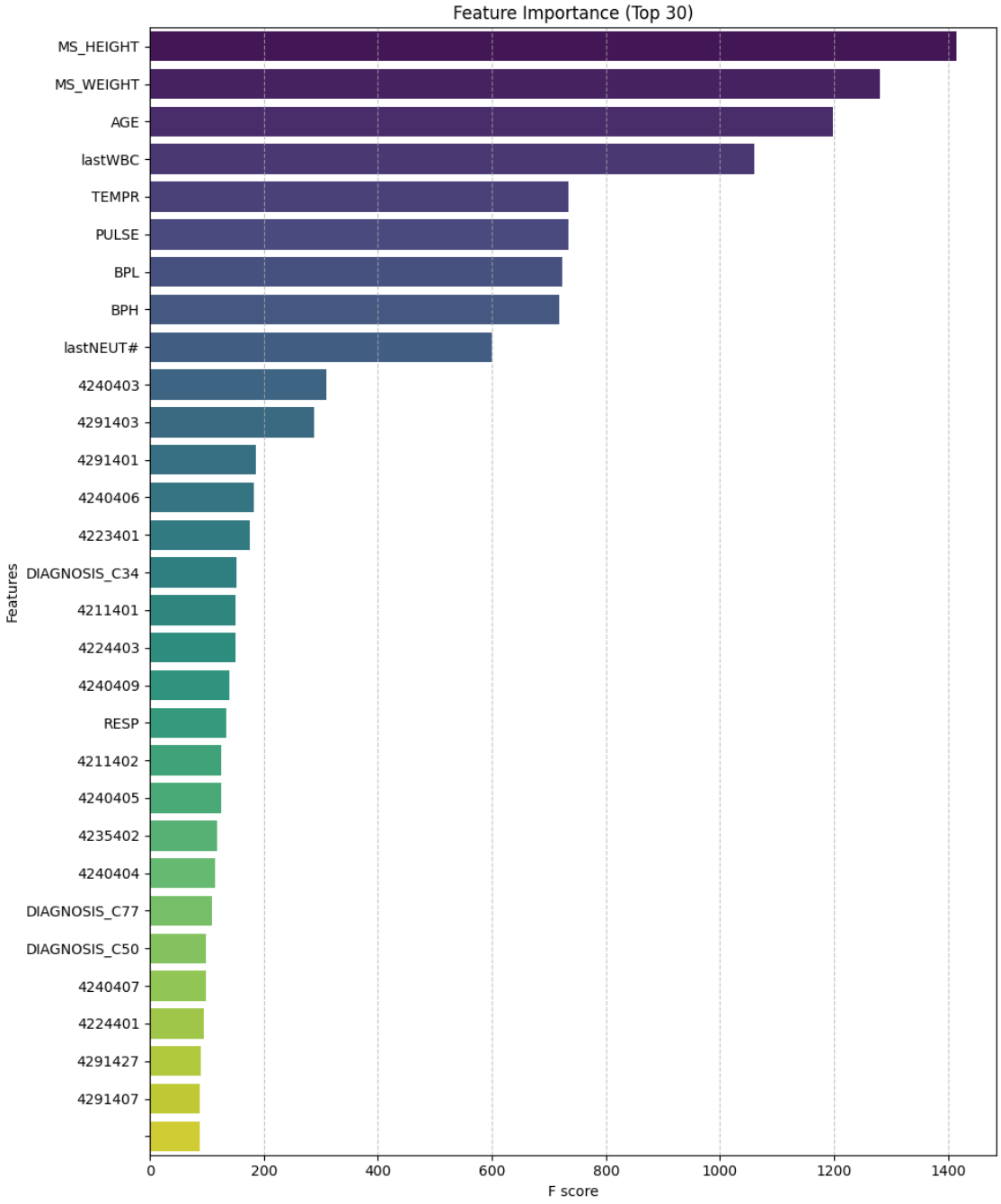
```
print("Summary Plot:")
shap.summary_plot(shap_values, X_test_scaled_df)
```

Summary Plot:



[87]:
```
# 1.      DataFrame
# get_score()      importance_type  'weight', 'gain', 'cover'
importance = xgb_model.get_booster().get_score(importance_type='weight')
df_importance = pd.DataFrame(list(importance.items()), columns=['Feature',␣
 ↪'Score'])

# 2. f0, f1...
#      feature_names
#    X_train_scaled_df
#   f0, f1...
feature_map = {f'f{i}': col for i, col in enumerate(X_test_scaled_df.columns)}
df_importance['Feature'] = df_importance['Feature'].map(feature_map)

# 3.
top_n = 30
df_importance = df_importance.sort_values(by='Score', ascending=False).
 ↪head(top_n)

# 4. Seaborn
plt.figure(figsize=(10, 12))
sns.barplot(x='Score', y='Feature', data=df_importance, orient='h',␣
 ↪palette='viridis')

plt.title(f'Feature Importance (Top {top_n})')
plt.xlabel('F score')
plt.ylabel('Features')
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()

#
plt.savefig('feature_importance_clean.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
Passing `palette` without assigning `hue` is deprecated and will be removed in
v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same
effect.
```


Feature Importance (Top 30)
MS_HEIGHT
MS_WEIGHT
AGE
lastWBC
TEMPR
PULSE
BPL
BPH
lastNEUT#
4240403
4291403
4291401
4240406
4223401
DIAGNOSIS_C34
4211401
4224403
4240409
RESP
4211402
4240405
4235402
4240404
DIAGNOSIS_C77
DIAGNOSIS_C50
4240407
4224401
4291427
4291407
Features
0
200
400
600
800
1000
1200
1400
F score

[ ]: